

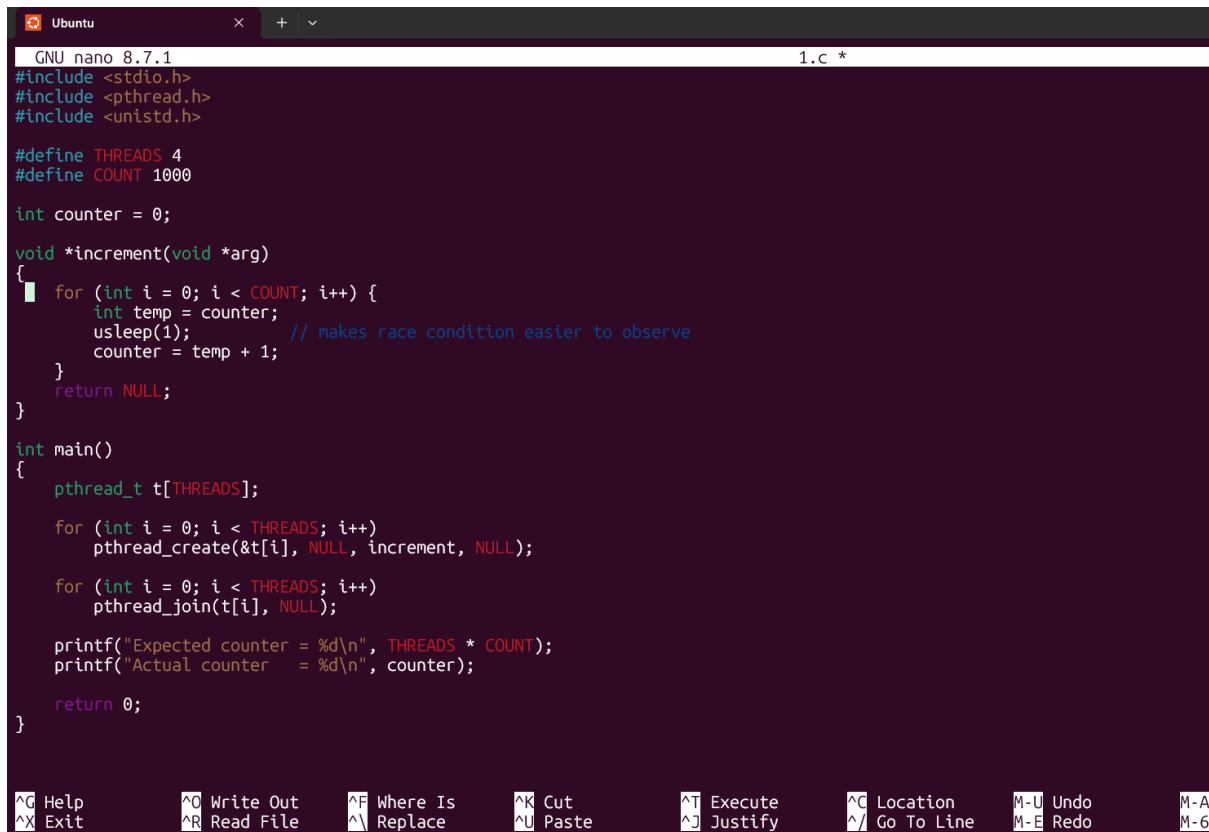
OSSP

Practical Week - 11

Roll No – 2520090085

Name - : Vakkalanka Sai Bhaskara Sujit

Task 1:-Develop a multithreaded counter application using POSIX threads (pthread_create, pthread_join). Demonstrate race conditions by allowing multiple threads to update a shared variable concurrently.



```
GNU nano 8.7.1 1.c *
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

#define THREADS 4
#define COUNT 1000

int counter = 0;

void *increment(void *arg)
{
    for (int i = 0; i < COUNT; i++) {
        int temp = counter;
        usleep(1); // makes race condition easier to observe
        counter = temp + 1;
    }
    return NULL;
}

int main()
{
    pthread_t t[THREADS];

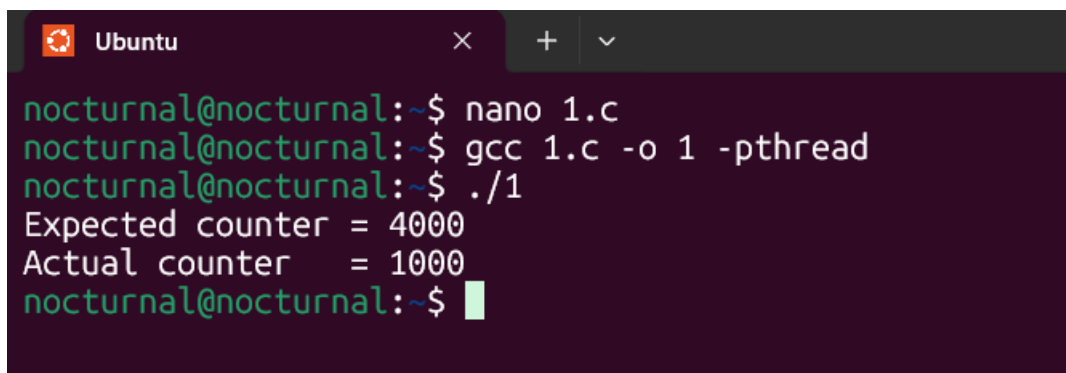
    for (int i = 0; i < THREADS; i++)
        pthread_create(&t[i], NULL, increment, NULL);

    for (int i = 0; i < THREADS; i++)
        pthread_join(t[i], NULL);

    printf("Expected counter = %d\n", THREADS * COUNT);
    printf("Actual counter   = %d\n", counter);

    return 0;
}
```

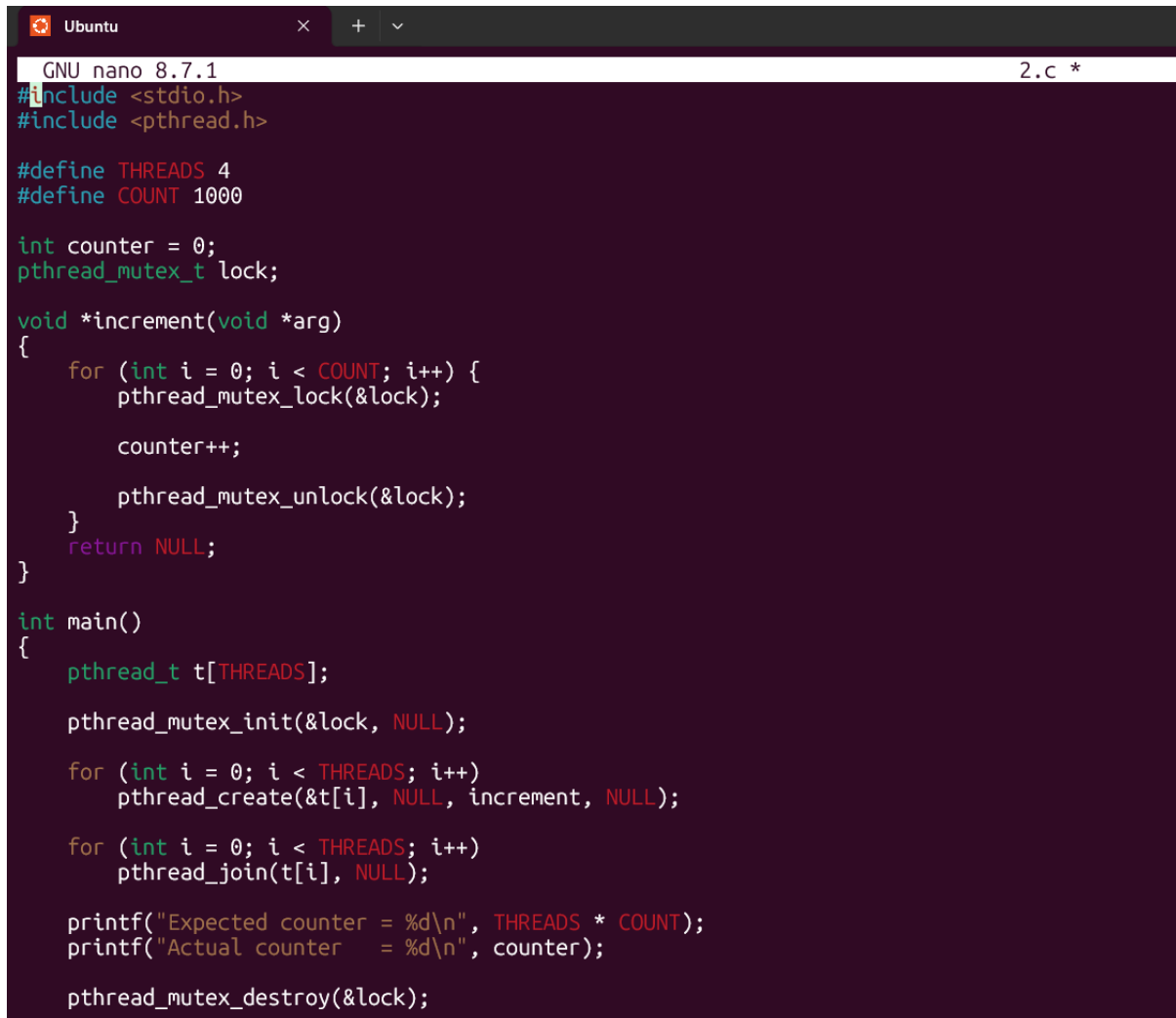
Output:-



```
nocturnal@nocturnal:~$ nano 1.c
nocturnal@nocturnal:~$ gcc 1.c -o 1 -pthread
nocturnal@nocturnal:~$ ./1
Expected counter = 4000
Actual counter   = 1000
nocturnal@nocturnal:~$
```

Task-2:

Modify the previous program using mutex locks (`pthread_mutex_lock`, `pthread_mutex_unlock`) and compare the outputs with the race-condition version. Analyze the impact of synchronization.

A screenshot of a terminal window with a dark background. The window title is 'Ubuntu'. The terminal shows the GNU nano 8.7.1 editor with a file named '2.c'. The code is a C program that uses pthreads to create 4 threads, each incrementing a shared counter 1000 times. It includes headers for stdio and pthread, defines THREADS as 4 and COUNT as 1000, and uses pthread_mutex_t for synchronization. The main function initializes the mutex, creates the threads, joins them, and prints the expected and actual counter values.

```
GNU nano 8.7.1 2.c *
#include <stdio.h>
#include <pthread.h>

#define THREADS 4
#define COUNT 1000

int counter = 0;
pthread_mutex_t lock;

void *increment(void *arg)
{
    for (int i = 0; i < COUNT; i++) {
        pthread_mutex_lock(&lock);

        counter++;

        pthread_mutex_unlock(&lock);
    }
    return NULL;
}

int main()
{
    pthread_t t[THREADS];

    pthread_mutex_init(&lock, NULL);

    for (int i = 0; i < THREADS; i++)
        pthread_create(&t[i], NULL, increment, NULL);

    for (int i = 0; i < THREADS; i++)
        pthread_join(t[i], NULL);

    printf("Expected counter = %d\n", THREADS * COUNT);
    printf("Actual counter   = %d\n", counter);

    pthread_mutex_destroy(&lock);
}
```

```
GNU nano 8.7.1
#define THREADS 4
#define COUNT 1000

int counter = 0;
pthread_mutex_t lock;

void *increment(void *arg)
{
    for (int i = 0; i < COUNT; i++) {
        pthread_mutex_lock(&lock);

        counter++;

        pthread_mutex_unlock(&lock);
    }
    return NULL;
}

int main()
{
    pthread_t t[THREADS];

    pthread_mutex_init(&lock, NULL);

    for (int i = 0; i < THREADS; i++)
        pthread_create(&t[i], NULL, increment, NULL);

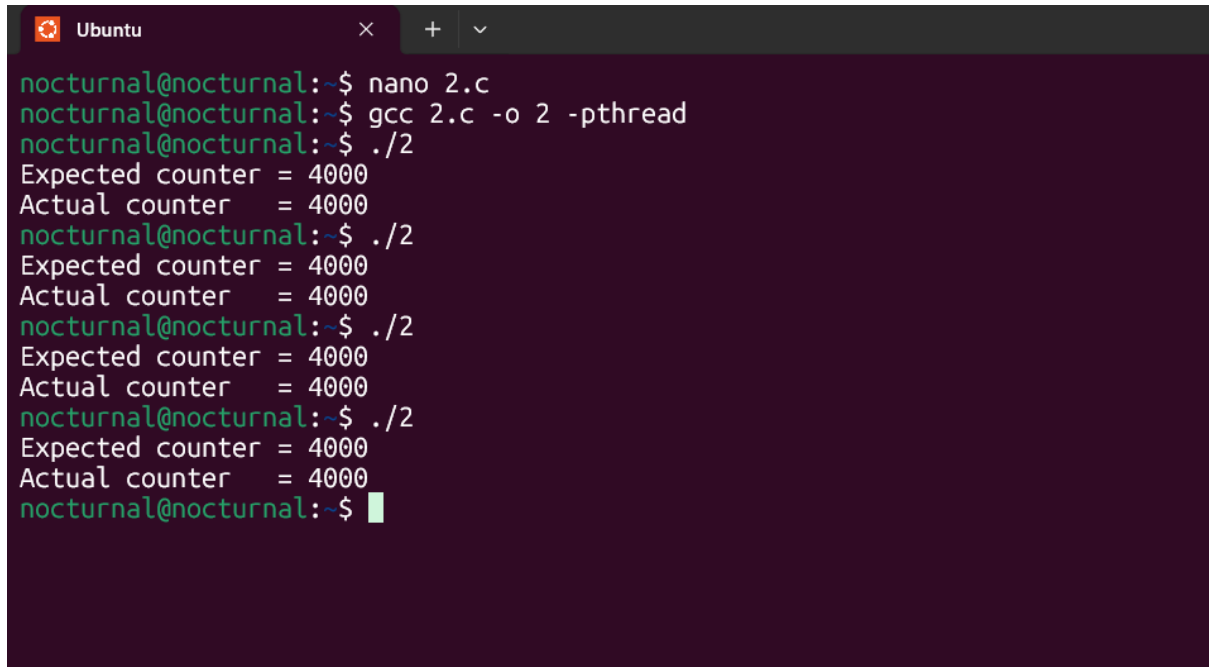
    for (int i = 0; i < THREADS; i++)
        pthread_join(t[i], NULL);

    printf("Expected counter = %d\n", THREADS * COUNT);
    printf("Actual counter    = %d\n", counter);

    pthread_mutex_destroy(&lock);

    return 0;
}
```

Output –



```
nocturnal@nocturnal:~$ nano 2.c
nocturnal@nocturnal:~$ gcc 2.c -o 2 -pthread
nocturnal@nocturnal:~$ ./2
Expected counter = 4000
Actual counter   = 4000
nocturnal@nocturnal:~$ ./2
Expected counter = 4000
Actual counter   = 4000
nocturnal@nocturnal:~$ ./2
Expected counter = 4000
Actual counter   = 4000
nocturnal@nocturnal:~$ ./2
Expected counter = 4000
Actual counter   = 4000
nocturnal@nocturnal:~$
```